A Structural Taxonomy of Player Experience in Game Design

First Pass

Registry: [HOWL-GAME-1-2026]

DOI: 10.5281/zenodo.zzz

Date: May 2026

Domain: Applied Philosophy

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

---

1. Purpose

This document enumerates the structural primitives of player experience in game design. It classifies what the player does, decides, learns, manages, and is gated by — not what they see, hear, or feel emotionally. Aesthetics (visual style, audio design, narrative content, mood) are out of scope. They matter for the total product but they are not gameplay.

The taxonomy follows a three-axis framework: mechanisms (what the game presents and what the player does), properties (contracts about experience quality), and principles (rules governing design assembly). This framework is adapted from infrastructure systems taxonomy [HOWL-INFRA-1-2026] and application architecture taxonomy [HOWL-COMP-9-2026], which demonstrated that the three-axis separation — mechanism, property, principle as distinct kinds of thing — prevents the conflation that makes design reasoning imprecise.

Game design vocabulary is in worse shape than infrastructure vocabulary. "Difficulty" names a mechanism (a difficulty selector), a property (the game is hard), and a principle (games should be appropriately challenging). "Progression" names a mechanism (an XP bar), a property (the player advances), and a principle (reward investment over time). "Balance" is used for everything. The qualification discipline from the infrastructure taxonomy — always say "difficulty mechanism" vs "difficulty property" vs "difficulty-related principle" — is needed here.

1.1 What This Taxonomy Is For

This taxonomy exists to support precise reasoning about player experience. Specific uses include:

Designing a gameplay system for a target player by selecting mechanisms whose properties match what that player needs. Tuning an existing system by identifying which mechanisms are miscalibrated and which properties are failing. Explaining why specific players bounce off specific designs by showing the mismatch between the game's structural profile and the player's cognitive budget, preferred loop level, or agency model. Comparing

games across genres by structural profile rather than surface content — the same way the application taxonomy showed that a healthcare records system and a project management tool can be structurally identical with different schemas.

1.2 What This Taxonomy Is Not

This is not a game review framework. It does not evaluate whether games are good or bad. It is not a psychology model — it acknowledges that player cognition matters but scopes to what the designer constructs, not what the player brings. It is not a development methodology — it says nothing about how to build game software. It is not a marketing framework — it says nothing about positioning, monetization, or audience acquisition.

1.3 The Aesthetic Boundary

The boundary test: if you could replace the entire aesthetic layer — every texture, every sound effect, every piece of music, every line of dialogue — and the game still plays the same way, still has the same challenge profile, the same decision space, the same pacing, the same mastery curve, then the aesthetic layer is out of scope and the remaining structure is in scope.

Dark Souls with different art is still Dark Souls as a gameplay system. The combat timing, the stamina management, the spatial reasoning, the punishment for mistakes, the bonfire checkpoint system, the enemy placement, the level design as gating — all of this survives a total visual reskin. What changes is mood, atmosphere, emotional coloring. Those are real and important for the total product, but they are not the gameplay.

Story content is similarly out of scope. Story is content that fills gating structure. A Fallout New Vegas quest could be "go kill 10 rats" or "investigate a conspiracy involving multiple factions" — the gating structure is identical. What changes is the quality of the content filling each gate, which affects engagement but not structural function. Narrative structure as a gating and delivery mechanism is in scope. Narrative content is not.

Feedback signals are in scope because the player needs information to make decisions. Whether that information arrives as a sound effect, a screen shake, a number, or an ASCII character is aesthetic. That the feedback exists and is timed correctly relative to the action it reports on is mechanical.

1.4 The Player as Component

The player is inside the system. In infrastructure, the user is outside — they interact through interfaces. In games, the player's cognition, skill, attention, and emotional state are inputs that the game's mechanisms operate on. A game that demands more cognitive tracking than the player can sustain will produce overwhelm regardless of how well-designed the individual systems are.

The taxonomy handles this by including attention as a resource (GM48) and cognitive budget fit as a property (GP04), but it does not attempt to model player psychology. The taxonomy describes what the game demands of the player. What the player brings —

their skill level, their tolerance for frustration, their preference for action vs planning — is the player profile that gets matched against the game's structural profile. The taxonomy provides the structural profile. Player profiling is a separate concern.

---

2. Structural Approach

2.1 The Three Axes

The taxonomy has three axes. Each axis contains a different kind of thing. Conflating them prevents precise comparison and inspectable decisions.

Mechanisms are units of work-doing within the player experience. A mechanism is what the game presents to the player and what the player does in response. A skill gate requires the player to execute at sufficient quality. A reaction loop operates at 0.2 seconds. A resource system tracks things the player accumulates and spends. Mechanisms are identified by function, not by implementation or aesthetic presentation.

Properties are contracts about experience quality. A property claims that something holds across play. Pacing claims that the rate of content, challenge, and reward delivery stays within a productive range. Attributability claims that player successes and failures feel caused by the player's own decisions. Properties are not mechanisms — they are qualities that emerge from mechanism configuration and calibration.

Principles are rules governing design choices. A principle constrains which mechanisms the designer selects, how they configure them, and how they combine them. "Gate for function not duration" tells the designer to ensure every gate serves a gameplay purpose. "Match information to consequence" tells the designer not to punish the player for things they couldn't have known. Principles do not perform work or make claims about specific guarantees.

2.2 The Triangle

The three axes relate to each other in a specific pattern:

Mechanisms provide properties. A well-calibrated gating system with appropriate traversal gates and gate release mechanisms provides the property of good pacing. A high-lethality consequence model combined with rich information signals provides the property of anticipatability.

Properties require mechanisms. The property of creative freedom requires authorial agency mechanisms plus sufficient building vocabulary plus system responsiveness. You cannot have creative freedom without mechanisms that let the player create and systems that respond to what they created.

Principles select among mechanisms and constrain how properties are realized. When multiple mechanism configurations could provide the same property, principles determine which configuration to choose. "Depth through interaction not enumeration" selects coupled systems over independent systems when the target property is depth.

2.3 Qualification Discipline

Several words in game design name both a mechanism and a property. The taxonomy requires explicit qualification whenever these words are used.

Progression as mechanism: XP bar, skill tree, equipment upgrade path. Progression as property: the player's experience of advancing in capability and access over time.

Depth as mechanism: system interaction that produces outcomes beyond surface presentation. Depth as property: the quality that more remains to discover or master beyond initial understanding.

Challenge as mechanism: a specific obstacle the player must overcome. Challenge as property: the quality that the game tests the player's abilities at an appropriate level.

Balance as mechanism: numerical tuning of system parameters. Balance as property: the quality that no single strategy dominates all others.

---

3. Mechanism Axis

Mechanisms are organized into families. Each family groups mechanisms by the role they play in player experience.

3.1 Gating (GF1)

Gating mechanisms require the player to demonstrate or achieve something before proceeding. Every gate has a threshold (what must be demonstrated), a release condition (when does the gate no longer apply), and may have modifier systems (ways to adjust the threshold).

Gating is also pacing and duration. These are not three separate things — they are three perspectives on the same mechanism. Gating is the designer's view: requiring the player to demonstrate X before Y becomes available. Pacing is the experiential view: the rate at which new content, challenge, and reward arrive. Duration is the economic view: how long the game lasts. A game that gates too aggressively feels slow-paced and padded. A game that gates too loosely feels rushed and shallow.

GM01 — Skill Gate Requires the player to execute an action at sufficient quality. The player knows what to do; the challenge is doing it well enough.

Sub-types: dexterity (timing and precision — Dark Souls dodge window, Quake aim), optimization (finding a good-enough solution in a large possibility space — RimWorld winter survival), execution under pressure (performing a known solution while other demands compete for attention — fighting a boss while managing stamina and positioning).

GM02 — Knowledge Gate Requires the player to possess specific information. The player may have the skill; they lack the knowledge of what to do or where to go.

Sub-types: location knowledge (where is the objective — find the red key), procedural knowledge (how to do something — know the crafting recipe), combinatorial knowledge (what goes with what — know the enemy's elemental weakness), contextual knowledge (what is the situation — know the NPC's background to pick the right dialogue option).

GM03 — Resource Gate Requires the player to spend or possess resources to proceed. The player may have the skill and knowledge; they lack the resources.

Sub-types: currency (pay gold to enter), consumable (need a lockpick), equipment (need a specific weapon or tool), capacity (need enough carry weight to bring the quest item back), permanent investment (need 50 STR — spent skill points cannot be recovered).

Skyrim's weight mechanic is a resource gate on the reward loop. The player completed the quest and killed the enemies, but the loot sitting on the ground is gated behind carry capacity. This creates a secondary optimization puzzle: maximize value per unit weight, or make multiple trips (trading time for reward completeness), or invest in carry weight enchantments (trading combat power for logistics efficiency).

GM04 — Comprehension Gate Requires the player to understand a system well enough to produce the target state. Puzzles are comprehension gates. The player is being tested on whether they grasp how the system works.

Sub-types: puzzle (put system into target state — Zelda dungeon mechanics, Portal physics), deduction (infer a rule from evidence — detective games, Obra Dinn), pattern recognition (recognize recurring structure — Baba Is You rule grammar, learning boss attack patterns as a comprehension exercise rather than a reflex exercise).

GM05 — Traversal Gate Requires the player to cover distance or navigate space. Traversal gates consume time and expose the player to content or hazards between the origin and destination.

Sub-types: full walking (continuous cost, never releases — Death Stranding, early Morrowind without transport), first-visit gated then released (walk there once, fast travel thereafter — Skyrim, Fallout), persistent soft gate (every trip has a reduced cost — GTA taxi, RPG carriage for a fee), no gate (instant teleport between any points — removes traversal as a mechanism entirely).

Death Stranding makes traversal itself a game by nesting a 0.2s balance loop inside the 300s destination loop. Walking demands continuous attention. Skyrim's traversal has no inner loop — you hold forward and the character moves. The traversal itself is passive until interrupted by a discrete encounter. This is why walking "always works" in Death Stranding and can feel like padding in Skyrim on repeat trips.

Progressive gate release is a common pattern: Skyrim releases discovery gating after first visit (fast travel unlocked), releases traversal time gating (fast travel is instant), but maintains a loot transport gate via carry weight that is itself partially releasable through character investment.

GM06 — Time Gate Requires the player to wait. The player may have skill, knowledge, resources, and comprehension — they must simply wait for the gate to open.

Sub-types: real-time (wall clock — mobile game energy timers), game-time (simulated clock — crop growth cycles, crafting timers), cooldown (action becomes available again after a delay — ability cooldowns, respawn timers), seasonal (content available only at certain game-time or real-time periods — holiday events, seasonal crops).

Time gates that serve no gameplay function other than extending duration or driving monetization violate gating discipline (GR01). Time gates that serve a pacing or strategic function (crop growth creates seasonal planning; cooldowns create rotation decisions in combat) are structurally valid.

GM07 — Discovery Gate Requires the player to find the question, not just the answer. Unlike knowledge gates where the player knows what they need and must find it, discovery gates require the player to realize that something exists to be found.

Sub-types: exploration (find a hidden area or item the game doesn't mark), experimentation (try combinations the game doesn't suggest — Breath of the Wild physics interactions), observation (notice something the game doesn't explicitly highlight — environmental storytelling clues, subtle NPC behavior changes).

Discovery gates are intentionally unsignaled. Signaling them would convert them into knowledge gates. The search itself is the gameplay.

GM08 — Social Gate Requires interaction with other players. Only present in multiplayer contexts.

Sub-types: group size (need N players to attempt content — MMO raid requirements), role requirement (need specific capabilities represented in the group — healer, tank), trust/reputation (need established relationships or standing — guild membership, trade reputation), competitive threshold (must outperform other players — PvP ranking requirements, auction house economics).

3.2 Temporal Structure (GF2)

Temporal structure mechanisms define the nested time loops at which play operates. Different games weight different loops. The player's cognitive mode shifts at each level. A game's temporal profile — which loops are active, which are dominant, how they nest — is a primary determinant of what kind of experience it provides.

GM09 — Reaction Loop (0.2s) Trained motor response to immediate stimulus. The player operates on pattern-matched reflexes, not deliberation. There is no time for planning at this level.

Sub-types: timing (hitting a precise window — Dark Souls dodge roll, rhythm game beat), precision (spatial accuracy — Quake railgun aim, cursor placement), rhythm (repeated pattern execution — combo strings, beat-matching).

Games dominated by this loop: Quake multiplayer, rhythm games, bullet-hell shooters, fighting games.

GM10 — Tactical Loop (2s) Immediate situation assessment and action selection. The player reads the current state and selects from a small set of available actions. Some deliberation occurs but is time-constrained.

Sub-types: threat assessment (which danger is most immediate), opportunity recognition (can I exploit this opening), micro-positioning (where should I stand right now relative to threats and objectives).

Pac-Man operates primarily at this level — continuously assessing ghost positions and selecting direction. Dark Souls melee combat alternates between 0.2s (dodge/parry execution) and 2s (should I attack, dodge, block, or reposition).

GM11 — Engagement Loop (10s) Completing a discrete unit of interaction. A single combat encounter, a conversation, a puzzle, a transaction. The player enters an engagement, works through it, and exits with a result.

Sub-types: combat encounter (fight begins, proceeds, resolves), conversation (NPC dialogue with choices), single puzzle (problem posed, solution found), transaction (buy, sell, trade, craft).

GM12 — Navigation Loop (60s) Moving between engagement points. This is where the "2-second rule" applies — moving in 2-second bursts should get you somewhere within this loop's timeframe. The player is oriented toward a destination and encounters things along the way.

Sub-types: pathfinding (choosing a route), landmark orientation (using known points to navigate), incidental encounter (things that happen along the way — random enemies, resource nodes, environmental details), route optimization (finding faster or safer paths).

Skyrim's typical play pattern operates heavily at this level. The player walks from one landmark to another, encountering wolves, picking ingredients, spotting a cave entrance. Each 60s navigation segment contains a few 10s engagements and occasional 2s tactical moments.

GM13 — Objective Loop (300s) Completing a quest phase, a dungeon, or a mission. Multiple engagement loops and navigation loops compose into an objective. The Fallout New Vegas quest loop lives here: get quest → go to place → solve problems → bring back evidence → receive reward.

This is the level where the player is pursuing a medium-term goal. They may navigate to three locations, fight through each, collect items, and return. The 300s timeframe means this is roughly a single play session's primary activity or one of two to three objectives completed in a longer session.

GM14 — Strategic Loop (3000s) Long-term planning and build decisions. Character build direction, faction alignment, base layout philosophy, campaign-level resource allocation. These decisions play out over many objective loops. The player revisits them infrequently but they shape everything below.

Skyrim level-up decisions live here. Dwarf Fortress fortress design philosophy lives here. Civilization technology path selection lives here. RimWorld's decision about whether to prioritize defense or production capacity lives here.

GM15 — Loop Nesting How loops relate to each other within a game. This is not a single mechanism but a structural property of how a game's loops are configured.

Nesting types: nested (inner loop demanded continuously during outer loop — Death Stranding's 0.2s balance within 300s traversal, making the outer loop active throughout), interrupted (discrete events break the outer loop and demand inner loop attention — RimWorld raid during base building, dropping from 300s authorial to 2-10s crisis response), empty (outer loop has no inner demand — Skyrim walking between fast-travel-discovered locations with no encounters, creating passive time).

A time loop is engaging when it contains an active inner loop. A 300s navigation loop with no inner loop is waiting. A 300s navigation loop with a 0.2s balance loop nested inside it is gameplay the entire time.

3.3 Information (GF3)

Information mechanisms determine what the player can observe, infer, or discover about the game state. The information model determines what kinds of challenge are possible. Full information allows only complexity, execution, and optimization challenges. Partial information adds discovery and information management as challenge sources.

GM16 — Full Observability The entire relevant game state is visible simultaneously. The player sees everything needed for every decision. There are no secrets — only the board, the player's resources, and possibly randomness.

Examples: Pac-Man (whole maze visible), chess (whole board visible), tactical combat games like Battle Brothers (all units, terrain, and stats visible during combat), Into the Breach (enemy intentions shown before player acts).

Full observability pushes all challenge into combinatorial complexity (too many possibilities to evaluate exhaustively), execution difficulty (you see what to do but doing it is hard), and optimization (you see all resources and demands but satisfying everything optimally is a hard problem).

GM17 — Partial Observability Some game state is hidden by design. The player must act with incomplete information, and information gathering is itself a gameplay activity.

Sub-types: spatial (fog of war, field of view limitations — Quake can't see behind you, RTS unexplored map), temporal (you can see the present but not the future — enemy

spawn timing, event scheduling), systemic (some systems are running but not displayed — NPC schedules, off-screen AI behavior).

Quake multiplayer is heavily defined by partial observability. Knowing where opponents are (through map knowledge, sound cues, item timing, and prediction) is a major skill axis. The game shows you a narrow field of view and you must construct the larger picture mentally.

GM18 — System Opacity Game systems exist that the player must discover or infer through play. The game does not explain all its mechanics — some must be learned through experimentation, observation, or external resources.

Sub-types: hidden mechanics (systems that operate but are not documented — damage formulas, AI decision logic), undocumented interactions (system combinations the game doesn't announce — Dwarf Fortress material properties affecting combat outcomes), emergent behaviors (outcomes that arise from system interaction in ways the designers may not have individually scripted — Breath of the Wild physics chain reactions), depth beyond UI (the simulation runs at higher fidelity than the interface exposes — Dwarf Fortress personality and relationship systems deeper than any single screen shows).

System opacity is what separates Dwarf Fortress from RimWorld structurally at the information level. Both have deep systems. Dwarf Fortress has more systems and exposes less about how they work through its interface. The player must discover mechanics through play, wiki research, or community knowledge. RimWorld surfaces more of its mechanics clearly, reducing the discovery gate but also reducing the depth-of-discovery experience.

GM19 — Feedback Signal The game communicates state changes to the player. This is the information delivery mechanism — how the player learns what happened and what is currently true.

Sub-types: immediate (damage numbers, hit confirmation, sound effect on impact — arrives within the reaction loop), delayed (quest log update, notification after an event resolves — arrives within the engagement or navigation loop), ambient (world state changes that aren't explicitly announced — weather shift, NPC behavior changes, resource node depletion), absence (lack of expected signal as information — silence where there was noise, enemy that didn't appear on schedule).

The aesthetic boundary applies here. Whether a hit confirmation is a red flash, a crunching sound, a damage number, or a screen shake is aesthetic. That the hit confirmation exists and arrives within 0.2s of the action is mechanical. The taxonomy cares about the timing, clarity, and information content of feedback, not its sensory modality.

GM20 — Causal Legibility The player can trace why something happened. Events have causes, and those causes are either visible, discoverable, or opaque.

Sub-types: short chain (A caused B — "I got hit because the enemy swung and I didn't dodge"), long chain (A caused B caused C caused D — "the colonist had a mental break because their friend died because the hospital lacked medicine because the trader was

killed by raiders last season"), illegible (outcome has no cause the player can trace — "something went wrong but I don't know why"), discoverable (cause exists but requires investigation — "I can figure out why this happened if I look at the right information").

RimWorld provides moderate causal legibility — mood breakdowns show their cause chain in the UI. Dwarf Fortress has longer causal chains and less UI support for tracing them, making legibility lower. Pac-Man has perfect causal legibility — everything that happens is immediately visible and caused by observable actions.

GM21 — Information Asymmetry Different parties in the game have different knowledge. This applies in multiplayer contexts and in single-player contexts where the player and the game's AI have different information access.

Sub-types: player vs game (AI knows things the player doesn't — enemy patrol routes, spawn timers), player vs player (competitive hidden information — poker hands, fog of war in multiplayer), temporal asymmetry (the player knows things now that they didn't know before — learned enemy patterns across attempts, meta-knowledge in roguelikes).

3.4 Agency (GF4)

Agency mechanisms determine how the player's intent translates into game state changes. The agency model is one of the strongest structural differentiators between game types.

GM22 — Direct Control Player input maps immediately to game entity action. The player moves, the character moves. The player presses attack, the character attacks. The translation from intent to action is immediate and specific.

Examples: Quake player movement and shooting, Pac-Man directional control, Dark Souls combat actions, Skyrim first-person movement and interaction.

Direct control games tie the player's reaction loop (0.2s) directly to game state change. The quality of the player's motor execution directly determines outcomes.

GM23 — Indirect Control The player sets priorities, rules, or goals, and autonomous agents execute within those constraints. The player's intent is mediated through AI behavior.

Examples: RimWorld colonist work priorities (the player sets "mining: 1, cooking: 2" and the colonist decides moment-to-moment what to do), Dwarf Fortress labor assignments and military schedules, auto-battle settings in strategy games, town management where NPCs carry out tasks.

Indirect control shifts the dominant loop upward. Instead of operating at 0.2s–2s executing actions, the player operates at 60s–3000s setting policies and observing results. The frustration mode unique to indirect control is when the autonomous agents make decisions the player disagrees with but cannot directly override at the moment it matters.

GM24 — Authorial Creation The player designs or builds persistent artifacts within the game world. The player's primary activity is bringing something into existence according to their vision.

Sub-types: freeform (relatively unconstrained placement — Minecraft creative building), constrained (grid-based or rule-based placement — SimCity zoning, factorio conveyor layouts), functional (the creation affects gameplay — RimWorld base layout determines defense effectiveness, Factorio factory layout determines throughput), decorative (the creation is primarily aesthetic — house decoration in Skyrim, cosmetic building).

The direction of intent is reversed compared to reactive gameplay. In Dark Souls, the game poses a problem and the player finds the solution. In SimCity, the player poses a vision and the game evaluates whether the player's execution achieves it.

Creative expression capacity equals authoring bandwidth available after disruption response is accounted for. SimCity (high authoring, low disruption) gives maximum expression. Dwarf Fortress (high authoring depth, high disruption frequency and depth) gives expression only to players whose cognitive bandwidth can cover both simultaneously.

GM25 — Selection and Commitment The player chooses from presented options with lasting consequences. Unlike direct control (which is continuous) or authorial creation (which is constructive), selection is discrete and often irreversible.

Sub-types: dialogue choice (pick a response that affects NPC relationships or quest branching), build path (choose a skill specialization that forecloses others), faction alignment (commit to a side with consequences for access and relationships), permanent upgrade (spend a resource on one improvement, precluding another).

The weight of a selection mechanism depends on its reversibility and its consequence magnitude. A dialogue choice that changes a single line of response is light. A faction choice that locks out 30% of game content is heavy.

GM26 — Timing Agency The player chooses when to act within a permissive window. The game doesn't force the action at a specific moment — the player decides the moment of commitment.

Sub-types: when to engage (choosing to start a fight vs avoiding it — Dark Souls approaching a new enemy group), when to retreat (choosing to disengage before losing — knowing when to run), when to use a consumable (popping the power pellet in Pac-Man, using a rare buff item), when to commit resources (launching a raid in RTS, spending accumulated currency).

This is where anticipation lives as gameplay. The Rainbow Six player staging near a corner is exercising timing agency. The game permits them to peek at any moment. Choosing the right moment — after gathering audio information, after predicting enemy position, after the flashbang detonates — is the gameplay. The game forces anticipatory play by making the consequence of bad timing (death) very high.

3.5 Consequence (GF5)

Consequence mechanisms determine what happens when the player succeeds or fails. The consequence model is one of the primary determinants of player behavior — high-consequence games produce anticipatory, careful play; low-consequence games produce impulsive, experimental play.

GM27 — Lethality Failure results in entity death or destruction.

Sub-types: instant (one-shot kill — Rainbow Six headshot, falling off a cliff), gradual (health depletion over multiple hits — most action RPGs), conditional (death only under specific failure modes — drowning only in water, fall damage only from height), cumulative (many small damages add up without recovery opportunity — attrition encounters).

Lethality interacts directly with temporal structure. High lethality at the 0.2s loop (one-shot kills) creates maximum anticipation pressure. Gradual lethality at the 10s loop (health bar depletes over an encounter) creates resource management during engagement.

GM28 — Resource Loss Failure costs accumulated resources without destroying the player entity.

Sub-types: partial (lose some of a resource — Dark Souls drops souls at death location, retrievable), total (lose all of a resource — some roguelikes clear inventory on death), specific (lose a particular valuable — breaking a rare weapon, losing a key item), permanent (lost resources cannot be recovered by any means).

Dark Souls' soul loss mechanic is precisely calibrated: you lose everything on death, but you can retrieve it if you reach your death location without dying again. This creates a secondary consequence loop — dying once is recoverable, dying twice is permanent loss. The retrieval run is itself a tension mechanism.

GM29 — Progress Loss Failure reverts the player's advancement.

Sub-types: checkpoint (return to most recent save point — Dark Souls bonfire, traditional save system), run loss (start the entire run over — roguelike permadeath), partial revert (lose recent gains but keep older progress — losing unsaved work, losing the last few minutes of exploration).

The severity of progress loss is measured in time-to-recover: how long does it take to get back to where you were? Dark Souls' bonfire system typically costs 1–5 minutes of replay. Roguelike permadeath can cost hours. The acceptable level of progress loss varies enormously by player and genre.

GM30 — State Persistence Consequences remain in the game world beyond the immediate event.

Sub-types: permanent death (an NPC killed is gone forever — Fire Emblem classic mode, Fallout NV killed NPCs), world change (environment altered by player action or failure — destroyed buildings stay destroyed), reputation (factions remember player actions —

Fallout NV faction system), degradation (equipment or resources wear down over time — weapon durability, food spoilage).

State persistence makes consequences compound. In a game without persistence, each failure is isolated. In a game with persistence, failures accumulate — lost NPCs mean fewer quest givers, degraded equipment means weaker combat capability, damaged reputation means closed faction content.

GM31 — Low Consequence Failure has minimal or temporary cost. The game deliberately cushions the player from significant loss.

Sub-types: free retry (immediate retry with no cost — many puzzle games, casual platformers), time-only loss (the only cost is the real-world time spent on the failed attempt), cushioned (the game prevents total loss through catch-up mechanics, generous checkpointing, or difficulty reduction on repeated failure), auto-recovery (the game restores lost resources automatically over time).

Low consequence is not inherently inferior design. It serves a different function — it encourages experimentation, reduces anxiety, and broadens accessibility. The tradeoff is reduced tension and anticipation, since the player has less at stake.

GM32 — Positive Consequence Success grants reward. This is the other side of the consequence model — what the player gains when things go well.

Sub-types: resource gain (currency, materials, consumables), capability gain (XP, stat increases, new abilities), access gain (new areas, new quests, new mechanics unlocked), information gain (lore, map data, system knowledge), narrative payoff (story progression, character development, plot resolution).

Positive consequences are the "pull" that motivates engagement with gating mechanisms. The gate says "demonstrate X." The positive consequence says "and you'll receive Y." The quality of the experience depends on whether Y feels proportional to the effort of demonstrating X.

3.6 Pressure (GF6)

Pressure mechanisms create external demands on the player's attention and resources. Pressure is what makes self-directed activity into a challenged activity.

GM33 — Disruption Pressure Discrete events that demand immediate response, interrupting whatever the player was doing.

Examples: RimWorld raids (drop everything, defend the colony), Dwarf Fortress forgotten beasts (immediate military crisis), random dragon attack in Skyrim (interrupts exploration), emergency events in SimCity (tornado, fire).

Disruption pressure interacts with authorial creation to produce the core tension of builder/management games. When disruptions test the player's construction (raids test your defenses), they feel purposeful — they validate the building activity. When

disruptions bear no relation to what the player built (random meteor on your base with no counterplay), they feel punishing.

The frequency of disruption is a primary calibration parameter. RimWorld calibrates raid frequency to colony wealth and development pace. SimCity rarely generates disasters (and they can be turned off). Dwarf Fortress generates disruptions from multiple independent sources (sieges, forgotten beasts, tantrum spirals, cave-ins, floods), making the aggregate frequency high even if each individual source is moderate.

GM34 — Constraint Pressure Continuous limits that shape ongoing decisions without discrete events. The player is never "attacked" by constraints — they are always present, always shaping what is possible.

Examples: SimCity budget (can't spend money you don't have), RimWorld map biome (tropical vs tundra determines available resources and threats), technology prerequisites (must research A before building B), population cap (can't have more than N colonists/citizens/units), map geography (mountains, rivers, coastlines constrain building placement).

Constraint pressure shapes expression without destroying it. Budget limits make the player prioritize. Map geography makes them adapt their vision to the terrain. Technology prerequisites create a progression structure within the authorial activity. These are the "interesting constraints" that make creative expression a puzzle rather than a free-form activity.

The structural distinction between disruption and constraint: constraints are continuous and shape expression; disruptions are discrete and challenge or destroy expression. Both are forms of pressure, but they produce different player experiences. A game of pure constraint pressure (SimCity with disasters off) is a creative optimization puzzle. A game of pure disruption pressure with no authorial activity (Quake deathmatch) is a reactive survival challenge.

GM35 — Cognitive Load The total mental demand across all active systems. This is a meta-mechanism — it's not a single system the designer builds but the aggregate effect of all systems on the player's mental capacity.

Components: system tracking (how many systems are active and need monitoring), state tracking (how much game state must the player remember), decision frequency (how often must the player make meaningful choices), causal depth (how far back in the causal chain must the player trace to understand current state).

Dwarf Fortress generates high cognitive load because all four components are high simultaneously — many systems, much state to remember, frequent decisions, deep causal chains. RimWorld reduces all four. The reduction in cognitive load is the primary structural difference between the two games, more than any individual mechanic.

GM36 — Time Pressure Real-time demands that limit the player's deliberation time.

Sub-types: continuous (clock always ticking, no pause — Quake, most action games), episodic (time pressure during specific events but not between them — RimWorld raids are real-time but base building can be paused), self-paced (player controls time flow — pause, speed up, slow down — most simulation/management games), turn-based (no time pressure within a turn — Civilization, XCOM, chess without clock).

Time pressure interacts with cognitive load to determine how much the player can think about each decision. High cognitive load plus high time pressure (Dwarf Fortress during a siege) is the most demanding combination. Low cognitive load plus no time pressure (turn-based puzzle game) is the least demanding.

GM37 — Escalation Pressure increases over time or with progress. The game gets harder, demands more, or introduces new pressure sources as the player advances.

Sub-types: linear (steady predictable increase — Pac-Man speed increase), stepped (discrete jumps at thresholds — new enemy types at specific levels), adaptive (responds to player performance — Left 4 Dead AI Director increasing or decreasing spawn rates based on player success), wave (cycles of intensity — build phase then attack phase, quiet period then crisis).

Escalation prevents mastery from making the game trivially easy. As the player gets better, the game gets harder. The rate of escalation relative to the rate of player improvement determines whether the game feels like a growing challenge (escalation slightly ahead of skill), a power fantasy (skill ahead of escalation), or a losing battle (escalation far ahead of skill).

3.7 Progression (GF7)

Progression mechanisms change the player's capabilities, knowledge, or access over time. Progression is what gives play a trajectory — the player's experience in hour 10 is different from hour 1 because things have changed.

GM38 — Character Advancement The player entity gains quantitative power through numeric increases.

Sub-types: stats (strength, dexterity, intelligence increase), skills (specific abilities improve with use or investment), levels (threshold-based power jumps), perks (qualitative capability additions at level thresholds), talent trees (branching specialization paths).

Character advancement is a resource gate modifier — as the player's stats increase, gates that were previously too difficult become passable. This is the core loop of RPG progression: encounter gate → cannot pass → grind/quest to advance → gate becomes passable → pass gate → encounter harder gate.

GM39 — Equipment Acquisition The player gains tools that modify capability. Unlike character advancement (which is typically permanent and incremental), equipment can be swapped, lost, found, and upgraded.

Sub-types: weapons (modify combat capability), armor (modify survivability), consumables (temporary capability boosts), utility items (enable new interactions — grappling hook, lockpick), crafted gear (player-created equipment combining found materials).

Equipment interacts with resource gates (need specific equipment to proceed), with carry capacity (can't carry everything), and with progression (better equipment enables harder content).

GM40 — Spatial Discovery The player's map and world knowledge expands. Areas that were unknown become known. Routes that were hidden become available.

Sub-types: map reveal (fog of war clears, showing terrain and landmarks), landmark discovery (specific points of interest found — Skyrim location discovery), fast travel unlock (discovered locations become teleportation targets), shortcut discovery (paths that reduce traversal cost — Dark Souls shortcuts that connect distant bonfires), area access (entirely new zones become reachable — unlocking a new region through story progression or ability acquisition).

Spatial discovery is a progression mechanism that directly modifies the traversal gate. Discovering a location releases its first-visit gate. Finding a shortcut reduces the traversal cost permanently. Unlocking a new area expands the accessible game world.

GM41 — Knowledge Accumulation The player learns game systems through play. This is player knowledge, not character knowledge — it persists across deaths, saves, and even separate playthroughs.

Sub-types: enemy patterns (learning boss attack sequences, enemy behavior rules), system rules (understanding how crafting, combat, or economy mechanics work), optimal strategies (discovering efficient approaches through trial and error), hidden mechanics (learning undocumented system interactions), meta-knowledge (understanding gained across runs that informs future runs — roguelike "I know this item is good because I used it last run").

Knowledge accumulation is the progression mechanism that makes games like Dark Souls and roguelikes work. The character may reset, but the player doesn't. Each death teaches something. The player's growing knowledge is the real progression, and it manifests as improved play on subsequent attempts.

GM42 — Access Expansion New content or systems become available as the player progresses.

Sub-types: zone unlock (new areas accessible after meeting conditions), ability unlock (new mechanics introduced at specific progression points), difficulty tier (harder versions of existing content become available), new game plus (entire game available at higher difficulty with carried-over progression), narrative branch access (new story paths open based on choices or completion).

GM43 — Gate Release Previously gated content becomes freely available. This is the inverse of gating — where gating restricts, gate release opens.

Sub-types: fast travel unlock (discovered locations become instant-travel destinations), permanent shortcuts (paths that remain open once found or created), ability-removes-gate (gaining an ability makes previously impassable obstacles trivial — Metroid missile doors), key items persist (quest items that permanently grant access).

Gate release is a progression mechanism because it changes the game's structure over time. Early-game Skyrim is heavily gated by traversal. Late-game Skyrim has released most traversal gates and the dominant gating has shifted to skill and resource gates for higher-level content.

3.8 Resource Systems (GF8)

Resource systems define what the player tracks, accumulates, spends, and manages. Resources interact with every other mechanism family — they gate progression, they are lost through consequences, they are spent at gates, they constrain expression, and tracking them contributes to cognitive load.

GM44 — Depletable Resource Consumed on use. Must be replenished from external sources.

Examples: health (depleted by damage, restored by healing), ammunition (consumed by shooting, found or purchased), consumable items (potions, grenades, lockpicks — used once and gone), food (consumed over time in survival/management games — RimWorld food stores), estus flask charges in Dark Souls (depleted by use, refilled at bonfire).

Depletable resources create a spending decision at every use. "Is this worth using now, or should I save it for later?" This decision is meaningful only when the resource is scarce relative to demand — abundant resources are effectively not gates.

GM45 — Accumulated Resource Grows over time through play activity. Spent on advancement, purchases, or gating requirements.

Examples: experience points (accumulated through combat/quests, spent on leveling), currency (accumulated through loot/selling/questing, spent on purchases), reputation (accumulated through faction-aligned actions, spent on faction access), research points (accumulated over time, spent on technology), building materials (gathered from the environment, spent on construction).

Accumulated resources are the primary fuel for progression mechanisms. The rate of accumulation determines how quickly the player can advance. Gating advancement behind accumulated resources converts time-spent into progression — this is the core of RPG and management game economies.

GM46 — Capacity Resource A bounded pool that constrains other activities. Not consumed — rather, it limits how much of other things the player can have or do simultaneously.

Examples: inventory space (limits how many items the player carries), carry weight (Skyrim — limits total weight of carried items), unit cap (RTS/management — limits how many units or colonists at once), building space (SimCity — limits total construction), action points (XCOM — limits actions per turn).

Capacity resources create optimization pressure. The player can't have everything, so they must choose what matters most. Skyrim's carry weight forces loot prioritization. XCOM's action points force tactical prioritization. RimWorld's colonist cap forces recruitment selectivity.

GM47 — Regenerating Resource Depletes through use and refills on a cycle, creating a rhythm of spend-and-recover.

Examples: health regeneration (recovers over time after damage), stamina (Dark Souls — spent on actions, regenerates when idle), mana or energy (spent on abilities, regenerates over time), cooldowns (ability becomes available again after a timer), respawning items (Quake weapon and health pickups reappear on a timer).

Regenerating resources create tempo within combat and engagement loops. Dark Souls stamina creates a rhythm of attack-then-wait. Ability cooldowns create rotation decisions — use ability A while B recovers. Quake item respawn timers create map control as a skill axis — knowing when items respawn and controlling those spawn points is a strategic layer over the moment-to-moment combat.

GM48 — Attention Resource The player's real cognitive capacity allocated across demands. This is the meta-resource. It is not tracked in-game, but it is consumed by gameplay. Every system the player monitors, every decision they make, every state they track draws from this finite pool.

This is what makes Dwarf Fortress harder than RimWorld at a structural level. Both games ask the player to manage colonies. Dwarf Fortress demands more attention across more systems simultaneously. When total attention demand exceeds the player's capacity, some systems go unmonitored, and failures emerge from the unmonitored systems. The feeling of being "overwhelmed" is attention resource exhaustion.

Attention resource interacts with every other mechanism family. More active systems (GF9) consume more attention. More active time loops (GF2) consume more attention. Less legible information (GF3) consumes more attention per unit of game state understood. Higher disruption frequency (GF6) consumes attention in spikes. The designer's control over attention demand is primarily through the number and depth of simultaneous systems, the legibility of those systems, and the frequency of attention-demanding events.

3.9 System Interaction (GF9)

System interaction mechanisms determine how the game's subsystems relate to each other. This family determines whether a game's depth comes from many independent systems (enumeration) or from fewer systems that interact richly (interaction).

GM49 — Independent Systems Game systems operate without affecting each other. Progress or failure in one system has no bearing on another.

Example: Skyrim's alchemy and smithing operate largely independently — improving one doesn't affect the other (setting aside enchanting loops that exploit their combination). Many mobile games have entirely independent progression tracks that share no resources or state.

Independent systems are cognitively cheap — the player can think about each one in isolation. But they produce shallow depth because there are no cross-system decisions. The player optimizes each system separately.

GM50 — Coupled Systems The output of one system feeds the input of another, creating chains of dependency and tradeoff.

Examples: RimWorld's food system produces meals → meals affect mood → mood affects work speed → work speed affects production of everything including food. This is a coupled loop. Factorio's entire game is coupled systems — iron plates feed into gears feed into assemblers feed into science packs.

Coupled systems create depth because decisions in one system have consequences in others. Allocating a colonist to mining means they're not cooking, which affects food supply, which affects mood, which affects productivity across the colony. The player must think across systems, not within them.

GM51 — Emergent Interaction Systems combine to produce outcomes that were not individually designed. The individual rules are defined, but their combination produces novel situations.

Examples: Dwarf Fortress fluid dynamics interacting with fortress design to create flooding defenses the developers didn't specifically design. Breath of the Wild's fire plus wind plus grass creating chain reactions. RimWorld animals going berserk after a player's hunting attempt goes wrong, leading to a cascade of injuries that overwhelms the hospital.

Emergent interaction is what produces the "stories" that Dwarf Fortress and RimWorld are famous for. No designer wrote "a dwarf will go insane because his favorite mug was destroyed in a siege and there's no replacement material because the caravan was killed by a forgotten beast." But the systems, interacting, produced that outcome.

GM52 — Feedback Loops System output reinforces or dampens its own input, creating self-amplifying or self-correcting dynamics.

Sub-types: positive feedback (success breeds success — rich-get-richer snowball, winning team gets more resources), negative feedback (success breeds difficulty — rubber-banding in racing games, harder enemies after leveling), stabilizing (extremes self-correct — ecosystem balance, market equilibrium in economic simulations), destabilizing (problems cascade — death spiral where wounded colonists can't fight, leading to more wounded colonists, leading to colony collapse).

Feedback loops are a critical design calibration. Unchecked positive feedback produces runaway winners and unrecoverable losers. Unchecked negative feedback prevents the player from ever feeling powerful. Destabilizing loops produce dramatic failure cascades that can be thrilling (barely surviving a crisis) or frustrating (unavoidable doom spiral).

---

4. Property Axis

Properties are contracts about player experience. They claim that something holds across play. They are not mechanisms — they are qualities that emerge from mechanism configuration and calibration. A property can be partially provided (holds in some contexts but not others), and each property explicitly does not claim things that might be confused with it.

Properties are organized into bands by what aspect of experience they address.

4.1 Engagement Band (PB1)

Claims about whether and how the game holds player attention.

GP01 — Pacing **Claim:** The rate of new content, challenge, and reward delivery stays within a productive range for the target player.

Conditions: Target player must be specified. Productive range means neither overwhelmed nor bored. Rate is measured relative to the player's active loop level — a 0.2s-dominant game paces through encounter density, a 300s-dominant game paces through quest structure.

Partial provision: Well-paced in early game but padded in late game. Well-paced in main quest but poorly paced in side content.

Does not claim: That the game is fun. Only that the delivery rate of content, challenge, and reward is calibrated to the target player's absorption rate.

GP02 — Tension **Claim:** The player experiences meaningful uncertainty about outcomes they care about.

Conditions: Requires stakes (consequence mechanisms create something to lose), uncertainty (information incompleteness or skill challenge makes outcome unpredictable), and investment (the player cares about the outcome because they've invested time, resources, or identity).

Partial provision: Tension in combat but not in exploration. Tension in boss fights but not in regular encounters. Tension in multiplayer but not in single player.

Does not claim: That tension is constant. Peaks and valleys of tension are part of pacing. Sustained maximum tension produces exhaustion, not engagement.

GP03 — Flow **Claim:** Challenge matches player skill closely enough to sustain absorbed engagement.

Conditions: Requires calibrated difficulty (challenge rises with skill), clear feedback (the player knows immediately how they're doing), and achievable-feeling goals (the next milestone feels reachable with effort).

Partial provision: Flow in core combat loop but not in inventory management. Flow for medium-skill players but not for beginners or experts.

Does not claim: That the game is easy. Flow can occur at very high difficulty if skill matches challenge. A Dark Souls player in flow state is intensely challenged but not frustrated.

GP04 — Cognitive Budget Fit **Claim:** The total cognitive demand across all active loops stays within the target player's capacity.

Conditions: Target player must be specified — different players have different cognitive budgets. All active loops must be counted — tracking, deciding, executing, remembering, and anticipating all draw from the same budget. Demand includes both sustained load (ongoing system monitoring) and spike load (crisis events).

Partial provision: Fits during normal play but overwhelms during crisis peaks. Fits for experienced players but overwhelms newcomers. Fits when the game is paused for planning but not during real-time execution.

Does not claim: That the game is simple. Only that the demand matches the audience. Dwarf Fortress provides cognitive budget fit for its target audience (players who enjoy high-complexity management). It does not provide cognitive budget fit for casual players. Neither is a design failure — they're different target audiences.

4.2 Fairness Band (PB2)

Claims about the relationship between player action and outcome.

GP05 — Attributability **Claim:** Player successes and failures feel caused by the player's own decisions and actions.

Conditions: Requires consequences to follow legibly from player action. Randomness, if present, must be within the player's risk assessment — the player chose to take a chance and it didn't work out, rather than being killed by something they couldn't have influenced.

Partial provision: Attributable in combat (I died because I dodged too late) but not in random loot (I didn't get the drop because RNG said no). Attributable for skilled players who understand the systems but not for newcomers who can't trace cause-effect.

Does not claim: That outcomes are deterministic. Only that outcomes are traceable to player action within the game's randomness model. A dice roll is attributable if the player chose to take the risk.

GP06 — Anticipatability **Claim:** The player can prepare for challenges if they attend to available information.

Conditions: Requires an information model sufficient for preparation — the game must provide signals the player can use to predict upcoming challenges. Requires a consequence model harsh enough to motivate preparation — if failure is costless, anticipation is pointless.

Partial provision: Anticipatable for repeat encounters (you've died to this boss before and now know its patterns) but not for first encounters. Anticipatable for players who read environmental cues but not for those who rush through.

Does not claim: That surprises never happen. Only that the game doesn't punish based on information the player couldn't have obtained. First encounters may surprise — the principle is that the game provides enough pre-encounter signals to make preparation possible, or makes first-encounter consequences low enough that surprise isn't devastating.

GP07 — Consistency **Claim:** Same game situations produce same ranges of outcomes.

Conditions: Rules don't change arbitrarily. Systems behave reliably across play. Exceptions are rare and signaled. The player can develop mental models that remain valid.

Partial provision: Consistent within a system (combat always works the same way) but not across updates or patches. Consistent for common situations but inconsistent for edge cases.

Does not claim: That outcomes are predictable. Complex system interactions may produce surprising-but-consistent results. A RimWorld colony collapse caused by a chain of events is consistent (each link followed the rules) even if the overall outcome was surprising.

GP08 — Proportionality **Claim:** The severity of consequences matches the magnitude of the player's error or the challenge's difficulty.

Conditions: Small mistakes incur small costs. Large mistakes incur large costs. Easy challenges give small rewards. Hard challenges give large rewards. The scale of investment matches the scale of return.

Partial provision: Proportional in combat (harder enemies give better loot) but not in instant-death traps (minor misstep causes total loss). Proportional in main quest rewards but not in side quest rewards.

Does not claim: That the game is gentle. Only that the consequence scale matches the error scale. Dark Souls is proportional — each death costs a bonfire run (proportional to the area's difficulty), not a full restart. Roguelike permadeath is arguably proportional at the run level (you lost the whole run because the run is the unit of play).

4.3 Comprehensibility Band (PB3)

Claims about whether the player can understand what is happening.

GP09 — Legibility **Claim:** The player can perceive the current game state with sufficient clarity to make informed decisions at their active loop level.

Conditions: Requires feedback signals matched to active loop level. Requires UI clarity sufficient for the decisions being made. Requires state representation that surfaces what matters and suppresses what doesn't at each level.

Partial provision: Legible for core combat systems but not for hidden subsystems. Legible through the UI but not through the game world itself.

Does not claim: That the game is transparent. Some opacity is intentional design — fog of war, hidden enemy stats, undiscovered mechanics. Legibility means the player can see what they need to see for their current decisions, not that they can see everything.

GP10 — Causal Clarity **Claim:** The player can understand why outcomes occurred.

Conditions: Requires traceable cause-effect chains at the depth the player needs for their current loop level. A 0.2s loop needs immediate cause-effect (I got hit because the enemy swung). A 300s loop may need longer chains (the colony is starving because the farmer broke down because their friend died in a raid).

Partial provision: Clear for immediate causes but opaque for systemic cascades. Clear in the UI's mood/cause display but opaque for deeper economic or social dynamics.

Does not claim: That every cause is shown. Only that causes are discoverable at appropriate effort. Some games make causal tracing itself a skill — understanding why things happen is part of the learning curve.

GP11 — System Learnability **Claim:** The player can discover how game systems work through play.

Conditions: Requires systems consistent enough to learn from (if the system behaves differently every time, no learning is possible). Requires feedback that confirms or denies player hypotheses (the player tries something and can tell whether it worked and why).

Partial provision: Learnable for core mechanics through play but requires external resources (wiki, guides) for advanced or hidden mechanics. Learnable for individual systems but not for system interactions.

Does not claim: That the game teaches explicitly. Learning through experimentation, observation, and failure is valid. A game that provides a comprehensive tutorial has high

learnability. A game that provides no tutorial but has consistent, learnable systems also has learnability — just with a higher initial investment.

GP12 — Depth **Claim:** More remains to discover or master beyond initial understanding.

Conditions: Requires system interaction or combinatorial complexity that exceeds surface presentation. The player's first-hour understanding is meaningfully incomplete relative to their hundredth-hour understanding.

Partial provision: Deep in core combat or management systems but shallow in side systems. Deep in theory but with a low skill ceiling in practice.

Does not claim: That depth is visible from outside. Hidden depth is still depth. A game that appears simple but rewards mastery with increasingly nuanced play has depth even if it doesn't look deep to a new player.

4.4 Progression Quality Band (PB4)

Claims about how the player's experience changes over time.

GP13 — Mastery Curve **Claim:** Player skill and knowledge grow in a satisfying trajectory from novice to competent to expert.

Conditions: Requires calibrated difficulty escalation that introduces new challenges testing new skills. Requires plateaus to feel temporary — the player should feel that continued effort will produce breakthrough. Requires the gap between current skill and current challenge to stay within a productive range.

Partial provision: Good mastery curve in combat but flat in crafting. Satisfying early-game learning curve but repetitive late-game.

Does not claim: That mastery is linear. Plateaus and breakthroughs are the normal shape of learning. A mastery curve that never plateaus is probably not demanding real skill development.

GP14 — Investment Return **Claim:** Time and effort spent translate into meaningful capability or access gain.

Conditions: Requires progression mechanisms that reward engagement proportionally. The player should feel that an hour of play made them meaningfully more capable, more knowledgeable, or more connected to the game's content.

Partial provision: Good return on main quest progression but poor return on grinding side activities. Good return in early game but diminishing returns in late game.

Does not claim: That all time spent is equally rewarded. Some activities can be their own reward (exploration for the joy of exploration). Investment return claims that the overall trajectory of time-to-capability feels worthwhile.

GP15 — Meaningful Choice **Claim:** Player decisions produce distinguishable outcomes the player can perceive and evaluate.

Conditions: Requires branching consequences that the player can observe diverging from what would have happened with a different choice. Requires the player to have enough information to understand what they're choosing between (even if not full information about consequences).

Partial provision: Meaningful faction choices in main quest but cosmetic dialogue choices in side quests. Meaningful build choices early when points are scarce but meaningless late when the player has invested in everything.

Does not claim: That choices are permanent. Reversible choices can still be meaningful if the experience of choosing and observing consequences was real. Respeccing a character build doesn't erase the experience of having played with the original build.

GP16 — Discovery Satisfaction **Claim:** Finding new content, mechanics, or system interactions feels rewarding.

Conditions: Requires things to find that the player didn't know about. Requires signals that discovery occurred (the player can tell they found something new). Requires novelty relative to what the player already knows.

Partial provision: Satisfying for major discoveries (hidden areas, secret mechanics) but routine for minor ones (another generic cave, another standard loot drop). Satisfying on first playthrough but not on subsequent ones.

Does not claim: That all content is hidden. Deliberately placed discoverable content is valid discovery. A hidden room behind a waterfall is designed to be found — the discovery is designed, and the satisfaction is designed, and that's fine.

4.5 Expression Quality Band (PB5)

Claims about whether the player can realize their intent through the game's systems.

GP17 — Creative Freedom **Claim:** The player's authorial intent can be meaningfully realized through game mechanisms.

Conditions: Requires authorial agency mechanisms (the player can create). Requires sufficient building vocabulary (enough tools, materials, and options to express different visions). Requires system responsiveness (the game's systems interact with what the player builds).

Partial provision: Free in structural building but constrained by available resources. Free in base layout but constrained by terrain and physics. Free in decorative choices but not in functional design.

Does not claim: That expression is unconstrained. Constraints can enhance creative expression by making choices meaningful. Minecraft survival mode is often more creatively satisfying than creative mode because resource constraints force creative problem-solving.

GP18 — Resilience of Expression **Claim:** Player-created artifacts survive disruption pressure long enough to be satisfying.

Conditions: Requires disruption frequency low enough or defenses strong enough that creations persist long enough for the player to appreciate and use them. The ratio of build-time to survival-time must feel worthwhile.

Partial provision: Resilient to minor events (small raids don't breach walls) but destroyed by major ones (massive siege flattens the base). Resilient in peaceful periods but vulnerable during difficulty spikes.

Does not claim: That creations are permanent. Only that the time between creation and destruction (if destruction comes) feels like a reasonable return on the investment of building. A wall that took 3 hours to build and survives 20 raids before falling in a massive siege has good resilience-of-expression. A wall that took 3 hours and is destroyed 5 minutes later by an unavoidable random event does not.

GP19 — Systemic Responsiveness **Claim:** The game's systems respond meaningfully to what the player builds or creates.

Conditions: Requires coupled systems that evaluate player-created artifacts against game mechanics. The player's creation must make a difference in how the game plays — not just exist as decoration.

Partial provision: Responsive to structural choices (wall placement affects raid pathing in RimWorld) but not to decorative ones (furniture placement doesn't affect combat). Responsive to large-scale design (city zoning affects traffic in SimCity) but not to small details.

Does not claim: That response is always positive. Negative response (your design failed, traffic is gridlocked, raiders breached the wall through the gap you left) is still systemic responsiveness and is often more valuable than positive response for driving learning and iteration.

---

5. Principle Axis

Principles are rules governing design choices. They select among mechanisms and constrain how properties are realized. Each principle includes its reasoning and notes counter-principles — situations where the opposite choice is valid. The existence of counter-principles is a signal that the domain requires judgment, not a sign that the principle is wrong.

5.1 Gating Discipline (PG1)

Principles governing how and when the game restricts the player.

GR01 — Gate for Function Not Duration **Rule:** Every gate should serve a gameplay purpose — teaching, testing, or pacing. No gate should exist solely to extend play time.

Reasoning: Gates that exist only for duration feel like padding. Players recognize and resent content-free time sinks. Walking back and forth between two NPCs who could have been placed near each other is duration padding. Walking across a dangerous landscape that teaches spatial awareness and tests survival skills is functional gating.

Counter-principle: Time gates in free-to-play games serve monetization. This is a valid business mechanism but violates this principle as a game design rule. Some walking segments serve atmospheric purposes, which is an aesthetic choice that works against pure gameplay engagement but may serve the total product.

GR02 — Match Gate to Demonstrated Need **Rule:** The type of gate should test what the player needs to demonstrate for the next content to be meaningful.

Reasoning: A skill gate before a skill-demanding section is coherent — the gate proves the player is ready. A fetch quest before a story revelation is incoherent unless the fetch quest teaches something the player needs for the revelation to land. Gates feel purposeful when they prepare the player for what follows. Gates feel arbitrary when they bear no relation to the upcoming content.

Counter-principle: Sometimes gates serve pacing even when the type doesn't match the upcoming content. A calm collection quest between two intense combat sequences serves the tension-and-release cycle even though the gate type (resource collection) doesn't match the upcoming content type (combat).

GR03 — Release Gates That Have Served Their Purpose **Rule:** Once a gate's function is fulfilled, continuing to impose it is cost without benefit.

Reasoning: Skyrim releases the traversal gate after first visit — you've seen the landscape, encountered the content, learned the route. Fast travel becomes available. Dark Souls opens shortcuts after you've mastered an area — you've proven you can get through, so the traversal challenge is released. The gate taught or tested what it needed to. Keeping it would be punishing mastery with repetition.

Counter-principle: Some games intentionally never release traversal gates. Survival games maintain travel cost as a persistent resource pressure. This is a genre-defining choice that keeps traversal meaningful throughout play. The principle applies most strongly to games where traversal is a content-delivery mechanism rather than a core system.

GR04 — Signal Gates Clearly **Rule:** The player should know a gate exists and roughly what it requires before investing significant effort toward it.

Reasoning: Invisible gates produce frustration because the player cannot plan. They invest time and effort toward an objective only to discover they can't proceed because of an unknown requirement. Visible gates produce anticipation and goal-setting — "I need

50 more gold to buy that weapon" gives the player a clear target. Signaled gates let the player make informed decisions about how to spend their time.

Counter-principle: Discovery gates are intentionally unsignaled. The search itself is the gameplay. This principle applies to progression gates (where the player is trying to advance), not to discovery gates (where the player is trying to explore).

5.2 Information Discipline (PG2)

Principles governing what the player knows and when they learn it.

GR05 — Match Information to Consequence **Rule:** Don't punish the player for things they couldn't have known or reasonably anticipated.

Reasoning: High-consequence actions require sufficient information for the player to make an informed choice. An ambush by a previously invisible enemy that kills in one shot violates this — the player had no information to act on and no chance to prepare. A boss with a telegraphed one-shot attack respects this — the player can see the windup and learn to avoid it.

Counter-principle: First encounters are inherently low-information. This principle requires either low consequence on first encounter (you can afford to die to a new boss because the bonfire is close) or sufficient pre-encounter signals (the game warns you through environmental cues, NPC dialogue, or enemy behavior that telegraphs before becoming lethal).

GR06 — Reward Observation **Rule:** Players who attend carefully to available information should gain advantage over those who don't.

Reasoning: If attentive play and inattentive play produce the same outcomes, the information system is decorative rather than functional. Audio cues in Rainbow Six should give an advantage to players who listen carefully. Environmental clues in Dark Souls should reward players who notice them. The information model exists to be used, and using it should matter.

Counter-principle: Some information should be subtle enough that only attentive players notice, creating a skill gradient in observation itself. The principle is that observation should be rewarded, not that all information should be equally obvious.

GR07 — Make State Observable to the Degree Needed for the Active Loop **Rule:** The player needs different information at different loop levels. Show what's needed for the current decision timescale.

Reasoning: A 0.2s reaction loop needs immediate visual and audio feedback — health change, hit confirmation, enemy position. A 300s objective loop needs a quest tracker, a map, and an inventory screen. A 3000s strategic loop needs a character sheet, a build planner, and a faction standing summary. Information appropriate for one loop level is

noise at another — showing strategic data during a twitch fight is clutter, and showing only moment-to-moment data during strategic planning is insufficient.

Counter-principle: Too much information at any loop level can reduce discovery satisfaction. Some players prefer to figure out the strategic picture without a build planner. Accessibility settings can address this — provide the information as an option rather than mandating its display.

GR08 — Legibility Scales with System Depth **Rule:** Deeper systems need more investment in legibility to remain comprehensible to the target audience.

Reasoning: Dwarf Fortress's depth combined with Dwarf Fortress's interface legibility is niche — only players willing to invest significant effort in learning the interface can access the depth. RimWorld's depth combined with RimWorld's clearer UI is broader — more players can access the (shallower but still significant) depth. The ideal is to simplify the interface without simplifying the system — make the depth accessible rather than removing it.

Counter-principle: Full legibility of very deep systems can reduce discovery satisfaction. Some opacity serves gameplay — figuring out how things work is part of the fun. The principle is that legibility should match the target audience, not that everything should be maximally legible.

5.3 Consequence Discipline (PG3)

Principles governing the cost structure of play.

GR09 — Consequence Proportional to Error **Rule:** Small mistakes should cost little. Large mistakes should cost much. Unavoidable situations should cost nothing.

Reasoning: Disproportionate punishment feels unfair — a minor misstep that causes total loss breaks attributability. Disproportionate leniency removes tension — if nothing matters, nothing is exciting. Punishing the unavoidable feels arbitrary and destroys the player's sense of agency.

Counter-principle: Some games intentionally use high consequence for small errors as a genre-defining choice. One-shot mechanics in tactical shooters create maximum anticipation play. The consequence is "disproportionate" by this principle's metric, but it serves a specific design goal (forcing careful, anticipatory play). The key distinction is that the high consequence applies to avoidable errors (you could have checked that corner) rather than unavoidable situations.

GR10 — Consequence Legible Before Commitment **Rule:** The player should understand what they risk before taking an action with significant cost.

Reasoning: Blind commitment to high-consequence actions feels like a gotcha — the player didn't know they were gambling everything. Informed commitment creates tension and meaningful choice — the player knows the stakes, weighs them, and decides to

proceed. “Should I enter this fog gate?” is meaningful when the player knows a boss is likely behind it. It’s a gotcha when the player doesn’t know and can’t save.

Counter-principle: Roguelikes intentionally obscure some consequences as part of the discovery model. Drinking an unidentified potion is a deliberate risk. The principle applies most strongly to irreversible high-cost choices.

GR11 — Provide Recovery Paths Proportional to Consequence Severity Rule: The harder the punishment, the more the game should provide ways to recover or avoid it.

Reasoning: Dark Souls loses your souls on death but lets you retrieve them — recovery path exists. It puts bonfires close to boss rooms — avoidance path (reduced repetition) exists. Roguelikes with permadeath give meta-knowledge that persists — each run teaches something for the next one. The principle is that harsh consequences should come with a way forward, even if that way forward requires skill or persistence.

Counter-principle: Some games intentionally provide no recovery as a difficulty commitment. Iron-man modes in strategy games, hardcore permadeath, no-save challenge runs. These are opt-in difficulty modifiers that the player accepts knowing recovery is unavailable.

GR12 — Couple Consequence to Expression When Expression Exists Rule: In games with authorial creation, disruption should test the player’s creation rather than randomly destroy it.

Reasoning: RimWorld raids test your defenses. The raiders try to get in, and whether they succeed depends on how well you designed your kill box, your wall placement, your turret coverage. The disruption validates the expression — you built well, so you survived. Or you built poorly, so you learn and rebuild. The consequence is coupled to the creation.

Random uncoupled destruction — a meteor lands on your base with no warning and no counterplay — invalidates expression rather than testing it. The player’s design quality was irrelevant to the outcome.

Counter-principle: Pure destruction events serve the fantasy of unpredictability in some games. Asteroid events in SimCity, random base destruction in some survival games. These are genre-dependent choices. The principle applies most strongly to games where expression is the core activity.

5.4 Tempo Discipline (PG4)

Principles governing the rhythm and timing of experience.

GR13 — Active Inner Loops Sustain Outer Loops Rule: Long-duration activities need continuous player engagement at a shorter timescale to remain interesting.

Reasoning: Death Stranding’s walking works because terrain balance is a 0.2s loop inside the 300s journey. Driving in GTA works because steering through traffic is a 2s loop

inside the 60s navigation. Skyrim's walking doesn't always work because there's no inner loop between encounters — the player is passively holding forward.

Counter-principle: Some games intentionally use empty traversal for atmospheric or contemplative purposes. Journey, Flower, some walking simulators. This is an aesthetic choice that works against pure gameplay engagement but may serve the total experience. The principle applies to games where traversal is meant to be engaging gameplay, not to games where traversal is meant to be meditative.

GR14 — Vary Loop Intensity Over Time **Rule:** Sustained high intensity exhausts. Sustained low intensity bores. Alternation sustains engagement over long sessions.

Reasoning: Action sequences need quiet aftermath for the player to recover, process, and appreciate what happened. Building phases need disruption punctuation to test what was built and break routine. The cycle of tension and release is structural — it's how long-form experiences maintain engagement without burning out the player.

Counter-principle: Some games commit to sustained intensity (Doom Eternal, bullet-hell games) or sustained calm (Stardew Valley, peaceful building games) as genre identity. These work because the intensity is matched to the audience's preference and the session length is appropriate.

GR15 — Interruptions Should Arrive at a Frequency the Target Player Can Absorb **Rule:** Disruption frequency should match the authorial or management loop it's interrupting.

Reasoning: Too frequent overwhelms — the player never gets to build or plan because they're always responding to crises. Too infrequent removes challenge and tension — the player builds without any test of their work. RimWorld calibrates raid frequency to colony wealth and development pace, keeping disruptions meaningful without making them constant.

Counter-principle: Adaptive difficulty systems (Left 4 Dead's AI Director) try to solve this dynamically by monitoring player performance and adjusting pressure in real time. This is a mechanism-level solution to the principle's calibration challenge.

GR16 — Self-Directed and Imposed Activities Should Be Coupled **Rule:** The things the player wants to do and the things the game makes them do should reinforce each other.

Reasoning: Building defenses (self-directed) is motivated by raids (imposed). Gathering food (imposed by hunger system) enables cooking skill advancement (self-directed). When self-directed and imposed activities are coupled, neither feels like a distraction from the other. When they're decoupled, imposed activities feel like interruptions and self-directed activities feel like ignoring the game's demands.

Counter-principle: Sandbox games sometimes intentionally decouple these, letting the player opt out of imposed activities entirely (turn off disasters, play on peaceful, use

creative mode). This serves players who want pure self-directed expression without challenge.

5.5 Complexity Discipline (PG5)

Principles governing how much the game demands of the player.

GR17 — Depth Through Interaction Not Enumeration **Rule:** Prefer fewer systems that interact richly over many systems that operate independently.

Reasoning: Chess has few piece types but deep interaction — the depth comes from how pieces relate to each other on the board. A game with 50 independent crafting materials has enumeration without depth — knowing 50 recipes is memorization, not strategic thinking. Coupled systems (GF9, GM50) produce depth through cross-system decisions. Independent systems (GM49) produce breadth through parallel optimization.

Counter-principle: Some games use large variety as content — Pokemon collection, item variety in ARPGs. Enumeration can serve completionist goals and discovery satisfaction even when it doesn't create strategic depth.

GR18 — Introduce Systems at the Rate the Player Can Absorb **Rule:** New mechanics should arrive when previous ones are understood, not before.

Reasoning: Tutorial pacing follows this principle when done well. Dark Souls introduces one new enemy type per area, letting the player learn its patterns before adding the next. Overwhelm from front-loading too many systems at once causes player abandonment.

Counter-principle: Expert players may find gradual introduction patronizing. Difficulty settings or experience-level selection can address this — let experienced players skip tutorials or access all systems immediately.

GR19 — Cognitive Demand Should Serve Gameplay Purpose **Rule:** Every system the player must track should contribute to meaningful decisions.

Reasoning: Tracking 12 resource types is justified if they create interesting tradeoffs (each resource has different acquisition sources, different uses, and different scarcity patterns, forcing prioritization). Tracking 12 resource types that all function identically is waste — the player is managing complexity that doesn't produce strategic decisions.

Counter-principle: Some cognitive complexity serves simulation fidelity or thematic flavor even when it doesn't create strategic depth. A survival game that tracks hunger, thirst, temperature, fatigue, and disease creates atmosphere through the management burden even if the optimal strategy for all five is similar.

GR20 — Reduce Depth for Breadth of Audience, Not by Removing Interaction **Rule:** When simplifying a game for a broader audience, preserve system coupling and remove system count rather than making systems independent.

Reasoning: RimWorld simplified Dwarf Fortress by having fewer systems that still interact richly. The food-mood-productivity loop is preserved with fewer food types, fewer mood modifiers, and simpler productivity calculations. The depth comes from the interaction, not the enumeration. Removing interaction (making systems independent) removes depth entirely. Removing systems (having fewer interacting parts) reduces complexity while preserving the structural quality that creates depth.

Counter-principle: Sometimes isolating systems serves accessibility by letting players engage with one system at a time without being forced into cross-system optimization. A cooking minigame that has no effect on combat lets the player enjoy cooking without worrying about combat implications. This trades depth for accessibility.

5.6 Expression Discipline (PG6)

Principles governing the player's creative and strategic agency.

GR21 — Constraints Enhance Expression **Rule:** Creative freedom is more satisfying with meaningful constraints than without them.

Reasoning: Minecraft survival mode is more creatively satisfying than creative mode for most players because resource constraints force creative problem-solving. "How do I build this with what I have?" is a more engaging question than "What do I build when I have everything?" Constraints make choices meaningful by forcing tradeoffs.

Counter-principle: Pure sandbox and creative modes serve a different need — relaxation, experimentation, pure authorial expression without challenge. This principle applies to expression-as-gameplay, not to expression-as-relaxation.

GR22 — Expression Should Be Testable **Rule:** The game should provide feedback on whether the player's creation works.

Reasoning: SimCity tests your city with traffic simulation and budget constraints. RimWorld tests your base with raids and environmental events. Factorio tests your factory with throughput demands. The test gives the authorial activity a gameplay loop — create, test, evaluate, revise. Without testing, expression has no feedback loop and becomes purely aesthetic.

Counter-principle: Decorative expression (house decorating in Skyrim, character cosmetics) is valid without functional testing. This principle applies to expression that is meant to interact with gameplay systems, not to expression that exists for personal satisfaction.

GR23 — Protect Expression Investment Proportionally **Rule:** The longer the player worked on something, the more the game should protect it from instant destruction.

Reasoning: A wall that took 3 hours to build shouldn't be destroyed in 3 seconds by a random event with no counterplay. The player's time investment deserves proportional

protection. Destruction should either be proportional to construction investment (a massive siege that overwhelms defenses built over many hours) or be preventable through player action (you could have built better defenses but chose not to).

Counter-principle: Some games use disproportionate destruction as a difficulty signature or as a thematic statement about impermanence. This is a genre choice that appeals to a specific audience.

---

6. Structural Elements Still Needed

This first pass establishes the three axes. Future passes need to develop the following structural elements to complete the taxonomy:

Double citizens. Words that name both a mechanism and a property, requiring qualification discipline. Identified so far: progression, depth, challenge, balance. The full enumeration needs to be completed with distinguishing definitions.

Spanning mechanisms. Mechanisms that genuinely belong to two families. Candidates: attention resource (GF8) spans into pressure (GF6); loop nesting (GF2) may span into pressure; feedback loops (GF9) may span into consequence (GF5). These need to be examined and resolved.

Orthogonality distinctions. Pairs that sound related but are structurally independent. Candidates: difficulty vs complexity (a game can be difficult but simple, or complex but easy); depth vs breadth (many shallow systems vs few deep systems); challenge vs frustration (challenge with sufficient information vs challenge without it).

Confusions. Commonly conflated pairs with distinguishing questions. Similar to the infrastructure taxonomy's confusions section. Candidates: pacing vs padding (is the gate serving a function?), difficulty vs punishment (how hard the challenge is vs how much failure costs), depth vs opacity (system interaction richness vs interface obscurity), progression vs power creep (player growth vs inflation).

The triangle in detail. Specific mappings of which mechanisms provide which properties, which properties require which mechanisms, and how principles select among mechanism alternatives. The infrastructure taxonomy's triangle section is the model for this.

Application layer — game type classification. Classification of game types (action, strategy, simulation, puzzle, management, survival, etc.) by their structural profile across all three axes. This is the equivalent of the application taxonomy's AC01–AC33. Each game type would be characterized by dominant mechanism families, critical properties, and governing principles.

Gap analysis. Where game design vocabulary diverges from structural reality. Candidates: "open world" (usually means large map with traversal gates, not genuinely open structure), "RPG elements" (usually means character advancement mechanisms grafted onto another genre), "roguelike" (widely applied to games with very different structural profiles), "difficulty" (conflates challenge, consequence, and complexity).

Fast locator. Symptom-to-family reverse index. "Game feels padded" → check gating calibration (GF1). "Game feels unfair" → check information-to-consequence match (GF3 vs GF5). "Game feels overwhelming" → check cognitive load (GF6) against target audience. "Game feels shallow" → check system interaction (GF9).

---

7. Summary Statistics

```
Mechanism families: 9

Mechanisms: 52

Property bands: 5

Properties: 19

Principle groups: 6

Principles: 23
```

This taxonomy is a first pass. The mechanism count, property count, and principle count will change as the taxonomy is refined. The structural claim — that mechanisms, properties, and principles are distinct kinds of thing and must not be conflated — is the load-bearing assertion. The specific contents of each axis are revisable. The separation is not.

[**HOWL-GAME-1-2026**] manuscript.md. Github: https://github.com/ghowland/papers/tree/main/papers/GAME/HOWL-GAME-1-2026

[**HOWL-INFRA-1-2026**] Infrastructure Taxonomy. Github: https://github.com/ghowland/papers/tree/main/papers/INFRA/HOWL-INFRA-1-2026

[**HOWL-COMP-9-2026**] Building Applications with OpsDB Application Architecture. Github: https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-9-2026